

Reflection

Computational Thinking

Ho Tuan Thanh

htthanh@fit.hcmus.edu.vn

thinking back and evaluating

Reflection

- ❑ Reflection (n): careful thought about something, sometimes over a long period of time.
- ❑ Reflective (a): thinking deeply about things.
- ❑ *Oxford Learners Dictionaries*

Purpose of reflection

- ❑ Identify strengths and weaknesses
- ❑ Derive lessons and improve solutions
- ❑ Develop long-term computational thinking skills

Principles of Reflection

□ Self-assessment

- Review the entire problem-solving process
- Evaluate effectiveness, efficiency, correctness, and creativity

Principles of Reflection

□ Feedback Integration

- Use evaluation results to improve solutions
- Reflection supports a continuous learning loop: Reflect → Refine → Reimplement

Principles of Reflection

- ❑ Meta-cognition: thinking about thinking
 - Awareness of how you think, analyze, and solve problems
 - Helps extract general strategies, reusable patterns, and best practices

Methods

□ Review / Retrospective

- *Think about or connect with something that happened in the past*
- Compare the solution with the original requirements
- Examine key decisions, algorithm choices, and representations

Methods

- ❑ Error Analysis / Debugging Reflection
 - Review errors, bugs, or failures in algorithms, decomposition, or simulations
 - Identify root causes

Methods

- Performance & Efficiency Reflection
 - Compare runtime, memory usage, and algorithm complexity against optimal solutions
 - Consider trade-offs between accuracy and efficiency

Methods

☐ Strategy Reflection

- Examine approaches: decomposition, abstraction, representation
- Ask: Is there a better way?
- Identify patterns that can be reused in other problems

Methods

- Documentation & Journaling
 - Record lessons learned, flows, decisions, and encountered challenges
 - Build a “knowledge base” for future reference

Tools

Purpose	Tools
Retrospective / Review	Notion, Confluence, Miro, Trello
Error & Debugging	IDE debuggers (VS Code, PyCharm, CLion), logs, profilers
Performance Analysis	timeit, cProfile, BenchmarkDotNet, MATLAB profiler
Journaling & Documentation	Markdown, OneNote, Google Docs, Obsidian
Visualization of Learning	Mind mapping tools: XMind, MindMeister, FigJam

Common Issues

1. Skipping Reflection Entirely

- ❑ Finish the problem → submit/close the laptop → never look back.
- ❑ No lessons learned, same mistakes repeated forever.

2. Superficial Reflection (“It works, done”)

- ❑ Never analyzing why one solution was better/worse, or realizing you over-engineered or under-engineered the problem.

3. Focusing Only on Correctness, Ignoring the Process

- ❑ "Got Accepted 100/100" → satisfied, but blind to the fact that your $O(n^2)$ solution barely passed while an $O(n)$ or $O(n \log n)$ existed

4. Not Keeping a Learning Log / Journal

- ❑ After 3–6 months you forget what mistakes you made and what tricks you discovered → zero long-term improvement.

5. Confirmation Bias in Reflection

- ❑ Only remembering what you did right and excusing or ignoring big mistakes (“the problem was tricky”, “I was running out of time”).

6. Never Comparing with Others' Solutions

- ❑ Skipping editorials and top coders' code → no idea where you stand or what elegant ideas you missed.

7. Vague, Non-Quantitative Reflection

- ❑ "My code felt a bit slow" instead of "My code was 15× slower because I used string concatenation in a loop".

8. Failing to Answer the Core Reflection Questions

- ☐ If I could redo it, what would I change?
- ☐ What was my dumbest mistake?
- ☐ Which pattern/abstraction did I miss?
- ☐ Was there a much simpler way?

9. Reflecting Only Once, Immediately After Finishing

- ❑ Reviewing again after a day or a week (when your mind is “cold”) usually reveals many more issues.

10. Reflection Without Concrete Action Items

- ❑ Writing “next time I’ll be more careful” but never creating checklists or deliberately practicing weak areas (e.g., always write brute-force first, always test $n=0$, $n=1$).

11. Confusing Reflection with Negative Self-Criticism

- ❑ Beating yourself up ("I'm so stupid") → demotivation instead of constructive, positive lessons.

12. “Nice Guy” Reflection in Team Settings

- ❑ Code reviews where everyone only praises each other and nobody points out real problems → the whole team stagnates.

Checklist

- ☐ How much time did I spend on each stage (decomposition, design, coding, debugging)?
- ☐ What was my most serious mistake?
- ☐ If I redo it, what would I change in strategy/abstraction/implementation?

Checklist

- ☐ Did I miss a simpler or more elegant solution?
- ☐ What specific takeaway do I have? (e.g., “always use long long in C++”, “always draw the segment tree before coding”)
- ☐ Which skill will I deliberately practice next week based on this?

